Aspis: A Lean-Checked Security Decomposition

for a Deployed Private-Spend Verifier

Dominic Barker

15 August 2026

Abstract

Aspis is a transparent private-spend verifier implemented as a Solana program. A recorded mainnet transaction verified a 75,358-byte proof, published its nullifier, and updated the pool state in 1,334,452 compute units.

The main contribution is a machine-checked security decomposition for that verifier. In one finite Lean 4 experiment, every accepted false statement is covered by one ideal-protocol event or one of eighteen named source, primitive, credential, and runtime events. Lean proves that this ledger has no duplicate entries, describes the same union as the earlier detailed ledger, and needs no independence assumption. The development also checks the private-spend relation after extraction, the released M31 circle domains and encoders, a prefix-timed four-fold FRI candidate argument, distinct-query sampling, transcript and five-tree Merkle models, theft reductions, and the spent-marker state transition.

As a conditional numerical consequence, the work-normalized protocol budget is at most $0.7 \cdot 2^{-100}$; if the remaining probabilistic terms total at most $0.3 \cdot 2^{-100}$, the combined budget is at most 2^{-100}. Separate raw accounting does not charge completed grinding work and leaves a 2^{-70} term plus explicit cryptographic and production terms. Focused Charon/Aeneas translations and a byte-for-byte reproducible Solana binary support the source and deployment connection. The main project-specific premise still to prove universally is that every accepting production Rust execution yields the exact Lean trace; published decoding and Fiat–Shamir results and standard primitive, toolchain, and runtime assumptions remain stated inputs.

1 Introduction

Succinct transparent arguments reduce a statement about a computation to a small number of algebraic and authenticated-opening checks. A deployed private-spend verifier adds another set of obligations. The proof must bind the exact public statement, the verifier must authenticate every value it uses, the nullifier must control a real state transition, and the implementation must preserve the order assumed by the Fiat–Shamir argument. Each part is simple to describe in isolation. Their composition is not.

Aspis brings that composition into one auditable development. It has three different kinds of evidence:

1. Lean definitions and proofs for the mathematical model;

2. focused Rust-to-Lean translations and source-shaped bridge theorems;

3. build and transaction records for the binary that ran on Solana.

Together they describe the mathematical verifier, selected production code, the compiled program, and one finalized execution. The report keeps the scope of each result clear so that future source proofs can be added without changing the mathematical statement.

The principal result is a checked security decomposition, not an unconditional claim of 100-bit deployed security. Every accepted false outcome in the modeled experiment lies in one ideal-protocol event or one of eighteen named events for source correspondence, primitive security, credentials, or runtime behavior. Lean proves that this compact ledger covers exactly the same union as the earlier detailed ledger and derives the combined probability bound in one sample space. This turns the remaining work into explicit proof obligations instead of hiding it inside one broad implementation assumption.

The mathematical background is kept short. Most of the report is devoted to the formal model, security experiment, proof reductions, and source locations. Lean declarations are cited by stable names instead of line numbers, which change during maintenance.

1.1 Contributions

The contributions reported here are:

- A machine-checked decomposition of accepted false outcomes into one ideal-protocol event and eighteen named external events. The old and new ledgers have a proved union equality, and the resulting conditional budget theorem uses one probability space without an independence assumption.
- A Lean proof of the released mathematical verifier: the complete one-input/one-output spend relation after extraction, exact M31 circle domains and encoders, four coefficient and verifier-weight folds, a prefix-timed four-round candidate argument, and the exact bounded distinct-query calculation.
- Byte-level transcript and five-tree Merkle specifications, together with source-shaped and focused Charon/Aeneas proofs that identify the exact remaining equalities for the enclosing production Rust functions.
- Fixed-victim and observed-history theft reductions, plus explicit state models for account checks, the non-overwritable spent marker, rollback, finalized persistence, and close/refund behavior.
- A reproducible source-to-binary record and archived mainnet execution evidence binding one program, proof, public statement, transaction, and before/after state.

To our knowledge, this is the first finalized Solana mainnet transaction in which a Solana L1 program directly verifies a trusted-setup-free proof of a note-based private-spend relation and, upon acceptance, atomically records the spent nullifier and advances the committed note state. This is a claim about that specific combination, not a claim that Aspis is the first private-payment system, shielded pool, or transparent verifier on Solana. Section 3 gives the comparison and its search date; Section 10 identifies the transaction, program, proof, public statement, and before/after state.

1.2 Scope

The 2^{-100} result is a conditional budget theorem for the work-normalized experiment defined in Lean. It is not the headline claim that deployed V5 has an unconditional 100-bit false-acceptance bound. The main project-specific work still open is the universal equality between an accepting production verifier run and the complete Lean trace. The published decoding and BCS Fiat–Shamir

Table 1: Entry points for the report.

Role	Lean item
Extracted trace implies the spend relation	`AspisV5AcceptedSpendRelation.extracted_trace_implies_spend_relation`
One adversarial experiment	`AspisV5UnifiedSecurityExperiment.WorkNormalizedV5AdversarialExperiment`
Conditional work-normalized budget theorem	`AspisV5UnifiedSecurityExperiment.work_normalized_accepted_false_probability_le_two_pow_neg_100_of_released_core`
Transcript schedule	`AspisV5TranscriptConnection.completeTranscriptTrace`
Authenticated five-tree forest	`AspisV5MerkleAuthenticationBinding.AcceptedV5Forest`
Production state transition	`AspisV5TheftStateTransitionReduction.runV5SourceTransition`

results, SHA-256 and Poseidon2 security, the proof assistant, compiler, and Solana runtime are stated as ordinary trusted inputs. The mathematical privacy and theft reductions are reported separately from a full production privacy or raw theft bound. Section A gives the complete list.

1.3 Reading guide

Section 2 describes the spend statement and fixed proof profile. Section 3 compares the result with proof-system, formal-methods, and Solana work. Section 4 presents the Lean model and its main definitions. Section 5 explains what is and is not connected to production Rust. Section 6 gives the single probability experiment and final conditional bound. Sections 7 and 8 cover theft resistance, state behavior, and privacy. Section 9 records proof choices that prevent several attractive but invalid shortcuts. The evaluation follows. The appendices collect assumptions, a dependency map, theorem index, failure ledger, and source-audit checklist.

2 Protocol background

2.1 The public statement and private witness

The public statement contains the pool address, sequence number, current note root, input nullifier, output note commitment, next note root, asset, fee, and deployment domain. The pool, sequence number, and deployment domain bind the proof to one state transition and one deployment. The six values consumed by the mathematical spend relation are the two roots, nullifier, output commitment, asset, and fee.

The private witness contains an input note opening, its ownership credential, the input note's fixed-depth Merkle path, and the data needed to form the output note and next root. Prover masks, Merkle salts, transcript salts, and retry randomness are proof-generation randomness, not part of the witness relation.

Write the input and output values as v_{in} and v_{out}, the public fee as f, and the public asset as a. The relation includes the integer equality

$$v_{\text{in}} = v_{\text{out}} + f, \tag{1}$$

Table 2: Fixed V5 parameters used by the formal model.

Quantity	Value
Base field	$\mathbb{F}_{2^{31}-1}$
Challenge field	degree-four extension $\mathbb{K}$
Trace rows	2^{10}
Initial evaluation symbols	2^{19}
Initial query fibres	$2^{17} = 131{,}072$
Batched lanes	19
Batch challenge degree	18
Queries	18 distinct fibres from at most 64 draws
Folds	four radix-four rounds
Final polynomial	four coefficients in $\mathbb{K}$
Work checks	batch, four folds, and final
Work difficulties	37, 34, 33, 30, 25, and 32 bits
Post-initial public-coin rounds	30
Authenticated private trees	five
Merkle record form	value bytes followed by a 32-byte salt

with all three quantities constrained below 2^{30}. The bound prevents a valid field equation from hiding an integer wraparound. Both notes carry the same public asset. Ownership is derived from a secret credential, the nullifier is derived from the credential and input salt, and each note commitment binds owner, value, asset, and note randomness.

Both note-tree paths have depth twenty. The input path reconstructs the current root from the input leaf. The output path uses the same path bits and siblings but substitutes the output leaf, producing the next root. This is a same-position leaf replacement. It is not a general append-only note-tree operation.

Definition 2.1 (Spend witness)**.** For deployed owner, note, nullifier, and node functions, a public statement has a spend witness when there are opened columns and integer input/output values that match the six public spend fields and satisfy the full `SpendRelation` predicate.

The exact Lean definition is `AspisV5AcceptedSpendRelation.StatementHasSpendWitness`. The more detailed `SpendRelation` is defined in `ArithmetizationCore` and includes the range, balance, ownership, note, nullifier, and two Merkle-root clauses.

2.2 The proof profile

The deployed V5 profile works over the Mersenne prime field $\mathbb{F}_{2^{31}-1}$, where $p = 2^{31} - 1$, and a degree-four extension field $\mathbb{K}$. It commits to sixteen base-field lanes and three extension-field lanes. A nonzero batching challenge combines these nineteen lanes with powers $\gamma^0, \ldots, \gamma^{18}$. The highest exponent is therefore eighteen, not twenty-eight as in an earlier experiment.

The first codeword is evaluated on a circle domain. Four circle symbols form one query fibre. Four radix-four folds reduce the initial word to three later line layers and then to four final coefficients. Each round also carries two out-of-domain values and relation messages. The verifier authenticates the two initial commitments and three later-layer commitments with five salted Merkle trees.

The protocol is related to STARK, FRI, circle-code, and multilinear polynomial work [8, 9, 14, 20, 21]. It is not an unchanged instance of any one cited protocol. In particular, its four-way fold,

Table 3: Background definitions and checked facts.

Topic	Lean item
Public statement and six-field match	`AspisV5AcceptedSpendRelation.V5PublicStatement`, `AspisV5AcceptedSpendRelation.OpenedColumnsMatchStatement`
Integer conservation	`value_conservation_no_wraparound`
Circle generator order	`AspisCircleGroupOrder.orderOf_g`
Released query sampler	`AspisV5TranscriptConnection.derive18Queries`
Five tree identifiers and widths	`AspisV5MerkleAuthenticationBinding.released_depths`, `AspisV5MerkleAuthenticationBinding.released_value_widths`, `AspisV5MerkleAuthenticationBinding.released_tree_tags`
State transition	`AspisV5ProductionStateBridge.runCurrentProductionSpend`

separate out-of-domain relation, bounded distinct-query sampler, work schedule, and mixed Merkle layout require separate arguments.

2.3 Transcript order

The transcript uses SHA-256 with typed labels. At a high level it absorbs the profile and statement, the initial roots and salts, public claims, relation points and messages, each later root and salt, the final polynomial, and six work nonces. It squeezes the batching challenge, relation challenges, four fold challenges, selector, and query material. Each work nonce is checked before it is absorbed, and the challenge protected by that check is squeezed afterwards.

The query selector is absorbed after the committed data, all relation messages, all four fold challenges, all work nonces, and the final polynomial have been fixed. The query sampler then reads 32-byte squeeze blocks as little-endian 32-bit words, masks them into the 2^{17}-element query space, keeps first occurrences, and succeeds only after collecting eighteen distinct positions within the 64-draw limit.

2.4 On-chain transition

The accepting instruction checks the proof before changing the pool. On success it creates or validates the spent-marker account derived from the public nullifier, writes the nullifier marker, updates the pool root and sequence, and returns success. A pre-existing marker is a rejection. The formal state model treats a rejected instruction as leaving visible state unchanged. A later close instruction refunds the retained proof account to the supplied refund account without undoing the spend.

The proof bytes are uploaded and finalized before the spend instruction. Thus the atomic claim concerns verification together with the marker and pool write. It does not claim that proof upload occurred in the same transaction.

3 Related work and position

3.1 Transparent proof systems

Aspis uses the IOP view of transparent arguments introduced by BCS and the FRI line of Reed–Solomon proximity tests [7–9]. DEEP-FRI strengthens proximity testing with out-of-domain samples, while WHIR combines proximity testing with multilinear evaluation claims [3, 10]. Circle codes and Circle STARKs provide smooth domains over M31, and S-two gives the closest published concrete circle-code protocol and BCS accounting [14, 20, 21].

Those papers supply the mathematical setting, but none specifies the V5 wire format. V5 has its own four-way fold, two initial commitment sections, three later Merkle sections, bounded distinct-query sampler, relation messages, and six work checks. The Lean development therefore proves the released domains, fold identities, prefix-conditioned bad sets, compatible four-round candidate chain, and adaptive fresh-attempt count for this schedule. It uses a published decoding theorem and the BCS Fiat–Shamir compilation instead of claiming new general versions of those results. The bounded-query treatment is also informed by later work on Fiat–Shamir security for FRI [12].

The distinction is important over a small field. Proximity-gap and mutual correlated-agreement results explain when folding preserves the required candidate structure [11, 13, 19]. Counterexamples to stronger Reed–Solomon proximity-gap conjectures show why a capacity-regime claim cannot be inferred from a Johnson-regime theorem [15]. Aspis records the exact released hypotheses rather than treating field size or code distance as a sufficient soundness proof by itself.

3.2 Machine-checked cryptography and verifiers

EasyCrypt established a practical language for machine-checked game-based security reductions [5]. The machine-checked MPC-in-the-head work goes further: it proves correctness and security for a general zero-knowledge construction and obtains executable implementations by extraction and verified Jasmin components [2]. Jasmin, HACL*, and Fiat-Crypto demonstrate strong source or generated-code guarantees for high-performance cryptographic primitives: Jasmin couples verified compilation with implementation proofs, HACL* uses F* to generate verified portable C, and Fiat-Crypto generates proved finite-field arithmetic [1, 17, 31]. These projects set a higher standard than tests for the code they cover; Aspis does not claim that its whole Rust verifier has yet reached that standard.

For proof systems specifically, ArkLib develops a modular Lean theory of interactive oracle reductions, composition, and knowledge soundness. Its published roadmap lists Rust equivalence for FRI and WHIR as future work [29]. Work on verifying deployed proof verifiers likewise emphasizes that a high-level security proof and a low-level verifier must be connected explicitly [18]. Aspis contributes a different point in this space: a maintained application-specific verifier, checked private-spend and state semantics, focused translations of selected production Rust, and exact outer equalities for the source still to be proved. Its main source result is therefore a precise decomposition, not a completed whole-program equivalence theorem.

3.3 Private payments and Solana verification

Zerocash established the note-commitment, nullifier, and private-spend pattern used by later shielded payment systems [6]. Several Solana projects provide important prior art on separate axes. Protocol 01 publishes a transparent STARK/FRI shielded-payment stack whose documented programs are on devnet; its inspected verifier and transfer paths separate proof phases from the later state transition

[27]. The solana-pqzk-fullchain project directly verifies a Winterfell STARK on Solana devnet and gives reproducible measurements, but its proved statement is a small affine-counter computation rather than a private spend [30]. Privacy Cash provides a mainnet shielded-state precedent with published Groth16 setup material; Light Protocol's published proving stack also records a Groth16 setup rather than a transparent proof system [24–26].

The comparison in Table 4 concerns the features needed for this paper, not overall product quality. "Checked" means a machine-checked security development, not merely a verified build or passing test suite. "Source link" distinguishes equivalence to executable code from a high-level protocol model. Aspis is marked partial in both columns because substantial Lean and focused source results exist while the enclosing production equalities remain open.

Table 4: Closest work along the axes combined by Aspis.

Work	Transp.	Private spend	State coupled	L1 evidence	Checked security	Source link
Zerocash [6]	no	yes	protocol	no	no	no
Machine-checked MitH [2]	varies	no	no	no	yes	generated code
ArkLib [29]	several systems	no	no	no	framework	planned
Protocol 01 [27]	yes	yes	split	devnet	no	no
solana-pqzk [30]	yes	no	no	devnet	no	no
Privacy Cash [25, 26]	no	yes	yes	mainnet	no	no
Aspis	yes	yes	atomic	archived mainnet	partial	selected; outer equality open

The public-evidence search was updated on 15 August 2026. It found prior work for every individual row of the comparison, but no artifact combining all of the Aspis row with a checked failure decomposition, reproducible deployed bytecode, and archived execution evidence. This supports the narrower contribution stated here. It does not support an absolute claim that Aspis is the first private-payment system, shielded pool, transparent verifier, or formalized proof system on Solana.

4 Lean formal model

4.1 Project organization

The maintained model is a Lean 4 project built on mathlib. It does not define one monolithic verifier function. Instead, it separates the spend relation, algebraic proof system, transcript, authenticated openings, source adapters, state transition, and probability accounting. The final theorems compose these pieces through named predicates. This makes an unfinished connection visible in a theorem type rather than burying it in prose.

There are two broad kinds of Lean files. Files in `AspisFormal/AspisFormal` define mathematical objects and source-shaped models. Files under `aeneas-verif` contain generated or supporting Lean for selected Rust extractions. The latter depend on Charon and Aeneas [22, 23]. A theorem about a pure source-shaped function is not described as a theorem about production Rust unless an extraction equality connects the two.

The principal dependency path is:

$$\begin{aligned}&\text{accepted production execution}\\&\quad\Longrightarrow \text{authenticated transcript and opened values}\\&\quad\Longrightarrow \text{one coherent low-degree candidate}\\&\quad\Longrightarrow \text{normalized arithmetic, hash, and Merkle rows}\\&\quad\Longrightarrow \text{a witness for the public spend statement.}\end{aligned}\tag{2}$$

Lean proves the final implication and substantial parts of the middle two. The first implication is split into focused source equalities and failure events; several outer equalities and probability bounds remain premises.

4.2 Arithmetic and spend relation

The structure `OpenedColumns` is the normalized algebraic view of the private spend. `ExtractedArithmeticResiduals` records the accepted residual equations for range, balance, asset equality, ownership, note creation, nullifier creation, and public fields. The theorem `ExtractedArithmeticResiduals.toConstraintsSatisfied` turns those equations into the higher-level arithmetic predicate.

Hash rows are represented more explicitly. A `TwoRoundPermutationRows` value carries one two-round piece of the Poseidon2 computation. Eleven such pieces form each checked permutation chain. `ExtractedMerkleLevel` and `ExtractedMerklePath` carry the per-level left/right placement and siblings for the two depth-twenty paths. The conversion functions construct the older abstract hash/Merkle witness; they do not assume that witness as an input.

Theorem 4.1 (Spend relation after extraction)**.** *Let O be normalized opened columns and let r_c be the fixed Poseidon2 round constants. If* `ExtractedV5Trace` *holds for r_c and O, and the deployed owner, note, nullifier, and node functions satisfy* `Poseidon2Faithful`*, then there are integer input and output values for which the complete* `SpendRelation` *holds.*

This is the Lean theorem `AspisV5AcceptedSpendRelation.extracted_trace_implies_spend_relation`. It derives both `ConstraintsSatisfied` and `HashMerkleWitness` from the extracted rows. It does not derive those rows from proof bytes.

The next definitions state the missing step without weakening it. A `V5PublicStatement` contains the exact public fields. The structure `OpenedColumnsMatchStatement` requires equality with all six spend fields. The predicate `AcceptedRunExtractsTrace` says that every accepted run outside a stated bad event yields matching opened columns and a nonempty `ExtractedV5Trace`. Given this predicate, Lean proves that false acceptance is contained in the same bad event. The measure theorem `false_accept_measure_le_bad_event` then turns set inclusion into a probability inequality, but supplies no numerical bound on its own.

Two regression theorems prevent a weak substitute. They show that invalid opened columns can fail the constraints and that success of a selected schedule check alone does not imply the trace constraints. Thus the missing extraction premise cannot be replaced by an unrelated Boolean check.

4.3 Field and circle domain

The base field is $\mathbb{Z}/p\mathbb{Z}$ for $p = 2^{31}-1$. `CircleGroupOrder` constructs the unit circle as a commutative group and proves that the released generator has order 2^{31}. From this it derives the criterion that two generator powers have the same x-coordinate exactly when their exponents agree up to sign.

The proof uses kernel reduction for the concrete repeated-squaring checks; it does not use compiled evaluation.

The extension field is represented as a fixed degree-four tower over M31. The deployment modules prove the packing basis and arithmetic identities used by the verifier. Separate arithmetic modules prove the raw M31 multiplication, subtraction, negation, and inversion addition chain used by compute-sensitive Rust.

The circle evaluation order matters because the released words are stored in bit-reversed order and grouped into radix-four fibres. The formalization proves:

- distinctness of the initial circle points and the later line-domain points;
- the exact correspondence between bit reversal and radix-four fibre positions;
- that the initial encoder evaluates its exact circle polynomial in the stored order;
- the exact polynomial-evaluation identities for the three later line encoders and final tensor encoder;
- later-code distances derived from exact agreement bounds of 255, 63, 15, and 3.

The published circle decoding result is represented by a proposition rather than reproved. The applicability audit substitutes the released field, distance, agreement threshold, multiplicities, width nineteen, degree eighteen, and nonzero challenge count into that proposition. The resulting theorem is therefore conditional on the cited result but not on informal parameter matching.

4.4 Four-round candidate chain

An accepted low-degree test must lead to one candidate that remains coherent through all folds. Choosing an unrelated decoder candidate after each challenge would multiply the list size and would not describe the protocol. The Lean development instead defines causal transcript families and backward selection functions. Later challenges may depend on completed earlier rounds, but the response strategy for a round is fixed by its preceding transcript.

`V5FriReleasedAdaptiveExtraction` defines one bad-challenge set per round, proves each cardinality bound, and bounds the four-dimensional tuple of adaptive bad challenges. The theorem `accepted_ideal_fri_extracts_initial_candidate_or_counted` states that an accepted ideal execution either enters one of those counted sets or yields one member of the initial decoder list whose exact four folds reach the published final coefficients.

The relation uses the same fold challenges. The candidate fold maps four values to one value, while the verifier folds the corresponding weights in the dual direction. `V5FriRelationCandidateBridge` proves that their dot product is preserved at every round. The relation soundness modules then show that false incoming or out-of-domain claims can be repaired by the sequential challenge mixes on only a bounded set of challenge tuples. The analysis is causal: a later claimed value may depend on earlier challenges, but not on a challenge that has not yet been sampled.

4.5 Transcript model

The transcript is represented as an ordered list of typed events, not merely as a list of abstract challenges. Absorb events contain a label and the exact byte payload. Squeeze events identify the requested field, point, selector, or query output. Work events contain the work kind, nonce, and difficulty. The complete schedule includes the profile and basis identifiers, public statement digest, all five roots and public salts, public claims, relation points, out-of-domain values, relation messages, the final polynomial, and the six work nonces.

Typed squeezes are expanded into primitive SHA-256 calls. An absorb advances the transcript state with one hash. A squeeze uses separate output and state advance hashes. A work check hashes its nonce without advancing the state; after the check succeeds, the nonce is absorbed by the next event. Lean proves exact event counts, uniqueness of operation slots, payload widths, and the immediate check-before-absorb ordering for every work nonce.

The source adapter decodes the fixed Rust body slices into transcript inputs and constructs the same pure event list. It proves that the exact derived values passed to later checks are γ, κ, the relation challenges, out-of-domain points and mixes, four fold challenges, selector, and eighteen query positions. The smallest remaining executable statement is `ExactRustV5TranscriptDriverEquality`: the production driver, after decoding, returns the same trace and consumed values as this source-shaped driver.

4.6 Authenticated openings

The Merkle model has five tree identifiers: the two initial commitments and three later layers. Each opened record is the value bytes followed by a 32-byte public salt. Leaf hashing uses a leaf prefix and tree identifier; binary and radix-four nodes use distinct prefixes. The child order and input lengths are part of the hash input.

The two initial sections use query index q. The later sections use $q \gg 2$, $q \gg 4$, and $q \gg 6$. Indices are sorted and deduplicated before frontier verification. Radix-four slot digits are taken from the shifted index. Because the released depths are odd, each path ends with a binary cap whose side is derived from the remaining index bit.

`ExactSectionTrace` records the parser and hash trace for one section. It accounts for every record, slot, frontier node, and returned remainder. The five-section driver passes each remainder into the next section and requires the last remainder to be empty. From an `ExactV5Run`, Lean constructs an `AcceptedV5Forest`. It then proves that every value read by the FRI schedule is the value part of an authenticated record.

The outer Rust connection remains explicit. The focused helper equality is `VerifyStateOnlyPrivateOpeningWithTopologySourceEquality`; the five-call driver equality is `VerifyV5DriverCompositionSourceEquality`. Together they imply the former whole-driver premise `VerifyV5PrivateOpeningsFromProofSourceEquality`. Outside the explicit SHA-256 collision event, `productionAcceptedForest_values_unique` proves that accepted forests under the same roots and queries expose the same values.

4.7 State model

The state model decodes exactly five spend accounts and checks account keys, owners, writability, pool state, sequence, anchors, marker availability, and the required marker-address bump. A successful transition writes the marker before the pool update and then returns from the callback. The current model requires bump 255. A pre-existing marker is rejected, including the case in which a different nullifier resolves to the same address.

The close model reads the refund destination from the supplied second account and transfers the complete retained proof balance. The composed trace places the spend writes before the later refund. Rejection rollback is a theorem of the explicit state model. Account locking, system-program behavior, and finalized persistence remain runtime premises rather than Lean facts about Solana.

Table 5: Principal Lean anchors for the mathematical model.

Claim	File and item
Extracted rows imply the spend relation	`V5AcceptedSpendRelation.lean`: `extracted_trace_implies_spend_relation`
False acceptance is in the extraction bad event	`V5AcceptedSpendRelation.lean`: `false_accept_event_subset_bad_event`
M31 circle generator order	`CircleGroupOrder.lean`: `orderOf_g`
Initial encoder identity	`V5FriInitialCircleEncoderIdentity.lean`: `encoder0_eq_stored_circle_eval`
Later encoder distances	`V5FriConcreteEncoderApplicability.lean`: `concrete_line_encoder_distances`
Released decoding applicability	`V5Width19S2ApplicabilityAudit.lean`: `published_appendixA2_supplies_exact_width19_curve_decodable`
Causal four-round extraction	`V5FriReleasedAdaptiveExtraction.lean`: `accepted_ideal_fri_extracts_initial_candidate_or_counted`
Candidate/verifier fold identity	`V5FriRelationCandidateBridge.lean`: `evaluation_discrepancy_eq_folded_difference`
Complete transcript order	`V5TranscriptConnection.lean`: `source_shaped_helper_composition_exact`
Source-shaped transcript adapter	`V5TranscriptSourceAdapter.lean`: `source_adapter_driver_is_exact`
Five authenticated sections	`V5MerkleRustBridge.lean`: `exactV5Run_yieldsForest`
Authenticated FRI reads	`V5MerkleConsumedValueBridge.lean`: `rustObservation_exposes_only_authenticated_fri_values`
Successful source state	`V5TheftStateTransitionReduction.lean`: `v5_source_success_exact_state`
Rejected transition rollback	`V5TheftStateTransitionReduction.lean`: `rejected_outcome_rolls_back_in_model`

5 Connection to the implementation

5.1 Production verifier path

The deployed verifier reads a fixed proof body, reconstructs the public statement digest, replays the transcript, verifies the six work records, derives the selected eighteen queries, authenticates five opening sections, checks the four FRI folds and final polynomial, checks the relation, and only then calls the atomic state transition.

The main production functions discussed in this report are:

- transcript state and SHA-256 framing in `crates/aspis-core/src/transcript.rs`;
- transcript replay, work checks, relation calls, and verifier control flow in `programs/aspis-verifier/src/v5_cu_probe.rs`;
- preparation and consumption of FRI values in `programs/aspis-verifier/src/v5_fri_checks.rs`;
- the five-section opening driver `verify_v5_private_openings_from_proof`;
- the focused opening helper `verify_state_only_private_opening_from_proof_with_topology`;
- account validation, marker creation, and pool mutation in `verify_and_apply_atomic_payment_state_traced_inner`;
- proof-account refund in `refund_program_owned_proof_account`.

The production Rust is larger than this list. The list identifies the path for which this paper gives formal anchors. Dispatch, upload, unrelated diagnostic modes, host retry code, compiler lowering, and runtime execution are not silently included.

5.2 Focused extraction

Charon translates selected safe Rust into its LLBC representation. Aeneas then produces pure Lean functions and proof obligations. This approach is well suited to small parser and arithmetic helpers, but broad Solana functions contain framework types, function pointers, syscalls, and control flow that are outside the convenient extraction subset. The project therefore uses focused extractions and source-shaped Lean functions for the remaining outer drivers.

For each connection we distinguish three statements:

1. a pure Lean model has the desired property;
2. the source-shaped composition equals that model;
3. the production Rust function equals the source-shaped composition.

Only the third statement turns the first two into a production theorem. When that statement is open, it appears as a named predicate.

5.3 Transcript coverage

The pure transcript result is detailed. It records every label and payload, expands each typed event to primitive hash inputs, places each work check immediately before its nonce absorb, and proves that the values supplied to the FRI and relation code come from the named squeeze results. The source adapter also decodes the exact fixed-body slices and exact selector byte.

The remaining boundary is the universal equality `ExactRustV5TranscriptDriverEquality`. Its arguments make the claim precise: decode one accepted Rust input into `V5TranscriptInputs`, decode the returned challenge values, and show that the real driver returns the same event trace and consumption record as `sourceShapedTranscriptDriver`. No theorem in the paper treats this equality as already proved for the full production driver.

The transcript hash itself is separated from hash security. The deterministic model says which bytes are hashed and in which order. The cryptographic model defines implementation-divergence, collision, preimage, transcript-divergence, and random-oracle events. Known-answer tests can support the implementation equality on selected inputs, but they cannot prove collision resistance or random-oracle behavior.

5.4 Merkle coverage

The source-shaped Merkle proof is split at two actual Rust functions. The focused helper predicate `VerifyStateOnlyPrivateOpeningWithTopologySourceEquality` says that the helper succeeds exactly when one encoded section has a valid `ExactSectionTrace` and the returned slice is the literal remainder. The outer predicate `VerifyV5DriverCompositionSourceEquality` says that the five-section Rust driver calls the helper in the fixed tree order, passes each remainder to the next call, and requires the final remainder to be empty.

Lean proves that these two equalities imply `VerifyV5PrivateOpeningsFromProofSourceEquality`. From that statement, successful production verification yields an `AcceptedV5Forest`. The forest contains one accepted path for every value consumed by the later FRI checks. Outside the explicit SHA-256 collision event, those values are unique under the supplied roots and query set.

This is materially narrower than assuming that "Rust acceptance yields a forest." The remaining work is attached to the two functions that must be extracted or otherwise proved equal. SHA-256 collision resistance remains a separate assumption even after those deterministic equalities are proved.

5.5 Relation and candidate coverage

The relation source model covers the prepared point claims, four unrolled round calls, staged claim updates, fold challenges, and final coefficients. Theorems in `V5RelationStressSourceBridge` show that the source-shaped caller implies the pure modeled relation checks and that its final coefficients equal the values supplied to FRI.

The remaining predicates identify exact production equalities rather than a general statement that Rust is correct. They include `ExactRustRelationVerifierEquality`, `ExactRustMode9RelationCallerEquality`, and the final raw-success connection. Other source modules prove the M31/QM31 arithmetic kernels, prepared-claim packing, row selection, and compact fold body separately.

The final candidate construction also has to connect authenticated values and transcript challenges to the eligibility, support, message, and response functions used by the correlated-agreement theorem. The mathematical theorem is universal over data satisfying that interface. The production mapping to that interface is still an external source obligation.

5.6 State-transition coverage

The state bridge gives a source-shaped description of the current spend and close functions. It proves exact account counts, refund-account selection, the bump-255 check, successful state writes, rejection of an existing marker, and agreement between current and recorded behavior at the observed bump.

The predicates `ExactCurrentRustStateEquality` and `ExactRustCloseEquality` identify the remaining Rust-to-model statements. The structure `ProductionRuntimePremises` then names account locking, system-program behavior, rollback, and finalized persistence. The theorem `runtime_premises_hold_outside_iff_no_named_failure` makes their failure events explicit. The model proves what follows if the runtime premises hold; it does not formalize the entire Solana runtime.

5.7 Source, binary, and chain identity

The deployed binary is identified by SHA-256 and size. A clean build from the recorded source and pinned tools reproduced the binary byte for byte. The release record then identifies the deployed program and finalized transaction. This establishes a useful chain of evidence:

$$\text{recorded source} \longrightarrow \text{recorded binary} \longrightarrow \text{recorded transaction.} \tag{3}$$

The first arrow depends on the build tools and the reproducibility record. The second depends on the loader and chain records. Neither arrow proves the cryptographic soundness of all accepted inputs.

The formal paper snapshot is `45d0ec466133bba6b7c52a39a288809a4943abf2` [4]. The clean source commit used for the deployed binary is `06788d44d30ea8cbd391899dddaf6f0acc6e4a3f`. They are intentionally reported as different objects: the formal development continued after the deployment.

6 Security decomposition and conditional bounds

6.1 Why one experiment matters

A list of individually small probabilities is not yet a security theorem. The events must be defined on the same probability space, false acceptance must be contained in their union, and each event must be charged once. Otherwise a bound can omit a branch, count one branch twice, or combine probabilities from incompatible experiments.

The module `V5UnifiedSecurityExperiment` gives the V5 endpoint this shape. It uses a finite probability mass function from mathlib rather than a new probabilistic programming language. The complete coins may include random-oracle answers, prover choices, and any other finite randomness needed by a concrete instantiation.

Experiment 6.1 (Work-normalized V5 adversarial experiment)**.** For a finite type Ω, a `WorkNormalizedV5AdversarialExperimentOmega` contains:

1. one probability mass function μ on Ω;
2. a set A_{false} of accepted false outcomes;
3. one record containing all named failure events; and
4. a pointwise proof that A_{false} is contained in their unified union.

The law is explicitly work-normalized. It is not the conditional law of an attempt after the adversary has completed the grind.

Existing accepted-execution reductions can construct this experiment through `ofFinalClassification`. The more specific constructor `ofReleasedAcceptedExecution` takes the maintained released-execution classification and its coverage proof. These constructors prove event inclusion; they do not create probability bounds for the included events.

6.2 Exact failure ledger

An older final ledger has twenty-four entries. Six belong to the ideal proof system: the query/final-work miss, four FRI rounds, and relation repair. The corrected arithmetic treats their union as one protocol event. The remaining eighteen entries describe implementation, primitive, credential, Merkle, and runtime failures.

Lean proves that the two lists are duplicate-free, disjoint, and concatenate to the old twenty-four-entry order. It then defines a nineteen-entry unified list: one protocol event followed by the eighteen external events. The theorem `total_unified_failure_eq_total_final_failure` proves equality of the two union events, not merely an implication.

The list is not a claim that all events have nonzero probability. It is the complete interface of the present endpoint. A later source proof can show that some event is empty or assign it budget zero. Until then, the event remains visible.

6.3 Deterministic premises and probabilistic events

The ledger has two uses that should not be confused. As an audit ledger, it keeps every way that production could depart from the mathematical experiment. As a cryptographic theorem, most of those departures are better stated as deterministic premises.

Let D_{prod} assert the exact transcript, SHA-256 implementation, Poseidon2 implementation, relation, Merkle, state, system-program, account lock, rollback, persistence, finalized-observation, and close/refund correspondences. These are entries 2, 3, 7, 10, 11, and 13–19 in Table 7. If D_{prod} holds pointwise for every reached execution, those twelve events are empty; they do not need an artificial tiny probability.

The remaining external entries are genuinely probabilistic assumptions: SHA-256 collision, target preimage, and random-oracle replacement; Poseidon2 collision and target preimage; and victim credential recovery. Write these six events as $P_1, \ldots, P_6$. Specializing the checked union bound with zero bounds for the deterministic events gives the conventional form

$$\Pr[A_{\mathrm{false}}] \leq \Pr[C] + \sum_{j=1}^{6} \Pr[P_j], \qquad \text{assuming } D_{\mathrm{prod}}. \tag{4}$$

The nineteen-entry ledger remains useful because it records exactly what must be proved before D_{prod} may be used. The security statement need not pretend that a parser mismatch or rollback error naturally has a 100-bit probability.

6.4 Symbolic union bound

Let C be the single protocol event and let $E_1, \ldots, E_{18}$ be the external events. From the pointwise classification and measure monotonicity, Lean proves

$$\Pr[A_{\mathrm{false}}] \leq \Pr[C] + \sum_{i=1}^{18} \Pr[E_i]. \tag{5}$$

Table 6: Production connections and their current status.

Area	Checked result	Remaining statement
Transcript	Exact pure schedule, primitive hash framing, six work checks, and consumed challenge mapping	`ExactRustV5TranscriptDriverEquality` for the enclosing production driver
Merkle helper	Exact parser, value/salt split, hash inputs, slots, binary cap, frontier use, and returned remainder	`VerifyStateOnlyPrivateOpeningWithTopologySourceEquality`
Merkle driver	Exact order of five sections and empty final remainder	`VerifyV5DriverCompositionSourceEquality`
FRI reads	Every source-shaped FRI read comes from an authenticated returned opening	Production equality for the enclosing opening/FRI consumer
Relation	Prepared claims, compact four-round evaluator, fold/weight algebra, and final coefficients	Exact equality for the outer Rust relation caller and its raw-success predicate
Spend state	Five account roles, bump 255, marker and pool writes, close/refund projection	Current Rust equality and Solana runtime premises
Hash primitives	Fixed inputs and known-answer executions	Concrete collision, preimage, and random-oracle bounds; all-input Poseidon2 equality

Table 7: The nineteen failure kinds in the unified experiment.

No.	Event	Meaning
1	Work-normalized protocol event	Union of the query/final-work miss, four FRI-round events, and relation repair
2	Transcript Rust-to-Lean divergence	Production transcript behavior differs from the exact Lean driver
3	SHA-256 implementation divergence	Reached SHA-256 call differs from the specified function
4	SHA-256 collision	Distinct reached inputs have one digest
5	SHA-256 preimage event	The adversary reaches a stated target digest
6	SHA-256 random-oracle event	The ideal-oracle assumption fails in the used reduction
7	Poseidon2 implementation divergence	Deployed and modeled Poseidon2 differ on a reached input
8	Poseidon2 collision	Distinct reached inputs collide
9	Poseidon2 preimage event	A stated target image is reached
10	Accepted-run relation bridge	Accepted production execution does not produce the modeled relation success
11	Proof/Merkle opening bridge	Production opening acceptance does not produce the authenticated forest used by the algebraic checks
12	Victim credential recovery	The adversary learns the victim's secret credential
13	Rust/state-model mismatch	Production state behavior differs from the Lean transition
14	System program or address mismatch	Account creation or address derivation differs from the model
15	Writable-account lock failure	Conflicting writes are not serialized as assumed
16	Rejection rollback failure	A rejected transaction leaves a visible partial state change
17	Marker persistence failure	A committed spent marker does not persist
18	Finalized-state observation failure	The observed finalized state does not match the modeled committed state
19	Close/refund mismatch	Proof close or refund differs from the modeled recipient and balance

This is `accepted_false_probability_le_core_plus_external_sum`. The theorem uses one PMF and maps the exact ordered list, so there is no separate independence assumption. It is an ordinary union bound.

Lean stores bounds for all eighteen external entries in `AssumedExternalSecurityBounds`. Its fields cover the transcript and primitives, relation bridge, Merkle bridge, credential recovery, and seven runtime events. The theorem `external_budget_sum_eq_total` proves that their sum is the declared external budget. Therefore

$$\sum_{i=1}^{18} \Pr[E_i] \leq B_{\text{external}} \tag{6}$$

follows from the individual assumptions without introducing a second core term. Under D_{prod}, the twelve deterministic fields are instantiated with zero and the budget is allocated only to the six probabilistic terms.

6.5 Query probability

The query calculation is one of the fully finite parts of the model. Each 32-byte squeeze block provides eight little-endian words. The sampler masks each word to seventeen bits, scans in order, and keeps only first occurrences. It accepts after collecting eighteen distinct positions and rejects if the 64-draw supply is exhausted first.

Conditioned on success, Lean proves that every ordered list of eighteen distinct positions has the same number of preimages. For a fixed bad set of size at most A in a domain of size N, the probability that every query lies in the set is

$$\prod_{j=0}^{17} \frac{A-j}{N-j}. \tag{7}$$

For the released calculation, $N = 131{,}072$ and $A \leq 6{,}082$. Without conditioning, the event "sampler succeeds and every output is bad" is bounded by the same ratio because exhaustion rejects.

Theorems in `V5BoundedQuerySamplerUniformity` prove the uniformity of the bounded first-occurrence sampler. The fixed-set ratio and its finite inequality are in `V5WithoutReplacementQuerySoundness`. These results do not show that SHA-256 outputs are independent uniform words, that the Rust sampler equals the finite model, or that an invalid FRI word determines the required bad set. Those statements occur in other event branches.

6.6 FRI and relation reductions

The low-degree part has three logically different steps.

First, the released encoders and distances must meet the hypotheses of the cited decoding theorem. Lean checks the field size, point distinctness, stored evaluation order, degree limits, distances, agreement thresholds, rounding, and list multiplicities. The decoding theorem itself is an external mathematical input.

Second, candidate selection must be causal and coherent. The forward theorem constructs four prefix-conditioned bad sets, each fixed before its challenge, and restores one initial list member through all four folds or enters one of those counted events. The list is not chosen again after each fold. The width-nineteen correlated-agreement argument handles a challenge-dependent family of committed lanes without adding an unjustified factor of 240 to the batching event. The adaptive fresh-oracle theorem then bounds repeated attempts without assuming that the bad sets from different attempts are independent.

Third, the candidate must satisfy the private-spend relation. The relation sumcheck theorem bounds the challenge tuples that can repair false scalar claims. The candidate/trace reduction lists the remaining ways in which a false no-witness statement could avoid such a mismatch: four-claim batching, lane recombination, public-field binding, arithmetic residual extraction, or hash/Merkle residual extraction. The production callback and authenticated opening connections remain separate external events.

The V5 four-way folding protocol is not identical to the published S-two protocol [14]. The project therefore proves its own prefix-timed four-way reduction in Lean instead of importing S-two's FRI reduction unchanged. Fiat–Shamir compilation is then supplied by the published BCS state-restoration result [7]; the production transcript equality and SHA-256 random-oracle assumption are kept explicit.

More precisely, the V5-specific chain theorem proves that ideal FRI acceptance restores one complete four-fold candidate chain or reaches a bad set fixed by the appropriate transcript prefix. The released accepted-false experiment then proves coverage by those FRI events and the relation events. Finally, the adaptive fresh-oracle theorem bounds Q completed fresh attempts by Q times the one-attempt bound, even when the next attempt depends on the entire previous history.

The remaining Fiat–Shamir connection is `ProductionAdaptiveFreshOracleConnection`. It must prove that the exact production byte transcript gives the ideal fresh attempts, counts repeated queries and rejection sampling correctly, and bounds the SHA-256-to-random- oracle difference. At the aggregate endpoint, `WorkNormalizedCoreEventConnection` identifies that production event with the protocol event used by the BCS bound. Neither interface asks for a new four-way FRI theorem. Closing it requires the enclosing Rust transcript equality and the published BCS/random-oracle assumptions.

6.7 Resource-bounded form

The same accounting can be written in the usual raw advantage form. Let $1 \le Q \le 2^{128}$ be the adversary's SHA-256 random-oracle query budget, let $R \le 32$ be the number of public-coin boundaries admitted by the released BCS accounting (the exact V5 schedule has 30), let ε_{I} bound each interactive branch, and use a 256-bit oracle output. The BCS expression represented in Lean follows the published state-restoration specialization [14]:

$$b(Q) = \left(1 + \frac{R}{Q}\right)\varepsilon_{\mathrm{I}} + 3\left(Q + \frac{1}{Q}\right)2^{-256}.$$

The factor three in the released ledger covers its three protocol branches. Multiplying by Q gives the ordinary resource-bounded statement

$$\mathrm{Adv}^{\mathrm{false}}_{\mathrm{V5}}(\mathcal{A}; Q) \le 3(Q + R)\varepsilon_{\mathrm{I}} + 9(Q^2 + 1)2^{-256} + \varepsilon_{\mathrm{prim}}(\mathcal{A}, Q) + \varepsilon_{\mathrm{cred}}(\mathcal{A}), \tag{8}$$

assuming D_{prod}, the decoding hypotheses, and applicability of the cited BCS theorem. Here $\varepsilon_{\mathrm{prim}}$ collects the explicit SHA-256 and Poseidon2 assumptions not already represented by the BCS capacity term, and $\varepsilon_{\mathrm{cred}}$ bounds victim-secret recovery. The paper does not assign concrete values to those two functions. Equation (8) is the standard companion to the work-normalized presentation; it makes both the query budget and the raw growth with Q visible.

6.8 Corrected numerical core

The corrected arithmetic uses nineteen batching lanes, highest exponent eighteen, the nonzero extension-field denominator, exactly thirty public-coin rounds after the initial committed data, and six separately positioned work checks. The dominant batching event has about 71 bits after the 37-bit grind has been completed. Charging for that grind produces a modeled core of about 100.56 bits. Lean proves the conservative inequality

$$B_{\text{core}} \leq \frac{7}{10 \cdot 2^{100}}. \tag{9}$$

The exact theorem is `corrected_selected_release_core_le_seven_tenths`. Its type retains the decoding, code-membership, grinding-output, Rust transcript, authenticated round, Merkle/PCS, Fiat–Shamir, and cited-theorem premises. Arithmetic does not discharge those premises.

6.9 Conditional budget theorem

The external budget is required to satisfy

$$B_{\text{external}} \leq \frac{3}{10 \cdot 2^{100}}. \tag{10}$$

The structure `WorkNormalizedCoreEventConnection` records the final application boundary: the exact event obtained from the production trace must be the event to which the BCS state-restoration bound is applied. The V5 four-way reduction used at that boundary is already proved. Discharging the structure requires the production trace equality and the stated BCS and random-oracle assumptions; it is not a further FRI decoding theorem.

Theorem 6.2 (Machine-checked conditional security composition)**.** *For a* `WorkNormalizedV5AdversarialExperiment`*, assume:*

1. *its accepted-false event has the proved pointwise classification;*
2. *the exact protocol event is connected to the released arithmetic;*
3. *the maintained applicability premises for that arithmetic hold;*
4. *the protocol budget satisfies Equation* (9)*;*
5. D_{prod} *holds, so the twelve deterministic mismatch events have probability zero; and*
6. *the six probabilistic external event bounds hold and satisfy Equation* (10)*.*

Then

$$\Pr[A_{\text{false}}] \leq 2^{-100}.$$

Lean proves this as `work_normalized_accepted_false_probability_le_two_pow_neg_100_of_released_core`. The proof is short because the substantive result is the preceding classification and exact ledger equality. This theorem checks that the stated protocol and external budgets compose to 2^{-100}; it does not independently prove those budgets for deployed Rust.

Remark 6.3 (Raw and work-normalized probability)*.* Dividing an attack probability by its expected work is useful for comparing attacks under a query/work budget. It is not the probability of failure once the work has already been paid. Removing the released 37-bit work charge restores a factor of 2^{37} to the dominant event. The raw completed-grind bound is therefore much weaker than 2^{-100}. No result in this paper describes the deployed verifier as having a raw 100-bit false-acceptance probability per completed proof.

Table 8: Soundness definitions and theorems.

Purpose	Lean item
One finite law and accepted-false event	`WorkNormalizedV5AdversarialExperiment`
Exact old/new ledger equality	`total_unified_failure_eq_total_final_failure`
One core plus eighteen external terms	`accepted_false_probability_le_core_plus_external_sum`
Exact external budget sum	`external_budget_sum_eq_total`
Bounded distinct-query uniformity	`AspisV5BoundedQuerySamplerUniformity.deployed_q18_conditioned_uniform`
Released prefix-timed four-round chain	`AspisV5FriForwardCompatibleChain.accepted_ideal_fri_restores_complete_chain_or_prefix_bad`
Adaptive fresh-attempt bound	`AspisV5ForwardAcceptedFalseRawAccounting.adaptiveCompatibilityReleasedFriFailureProbability_le_queryBudget_mul`
70-percent core budget	`AspisV5HundredBitSecurityMargin.corrected_selected_release_core_le_seven_tenths`
Conditional 100-bit budget implication	`work_normalized_accepted_false_probability_le_two_pow_neg_100_of_released_core`

7 From false acceptance to theft

A false-acceptance theorem is necessary for theft resistance, but it is not the whole claim. Theft also depends on how a victim note is chosen, what the attacker may observe before acting, how note and nullifier hashes are used, and whether the chain records a successful spend exactly once. The Lean development treats these questions separately.

7.1 A fixed victim

The simplest game fixes one live note, its depth-20 position, and its Merkle siblings before the attacker acts. The target contains a valid opening $(k_\nu, r_{\text{in}}, v_{\text{in}}, a)$. Its public note commitment, anchor, and nullifier are computed with the deployed owner, note, nullifier, and node functions.

The event `FixedVictimAttackEvent` covers two ways of targeting that note. An accepted proof either uses the victim's nullifier, or it presents a leaf at the victim's exact position under the victim anchor. This definition also includes the honest owner. In a security game, an honest opening is therefore classified as credential recovery rather than silently excluded.

Lean proves the following complete case split.

Theorem 7.1 (Fixed-victim classification)**.** *Every accepted fixed-victim attack implies at least one of:*

1. *extraction failed to produce a witness satisfying the spend relation;*
2. *the extracted witness contains the victim's secret and nullifier randomness;*
3. *a different secret/randomness pair has the victim's nullifier;*
4. *a different valid note opening has the victim's input commitment; or*
5. *a different leaf at the victim's exact tree position reaches the victim anchor.*

In the last case, the two paths expose distinct ordered node inputs with the same node-hash output at some level.

The pointwise result is `fixed_victim_attack_implies_mathematical_failure`. Its set form is `fixed_victim_attack_subset_mathematical_failures`, and `fixed_victim_attack_measure_le_five_failures` gives the corresponding union bound. None of these theorems assigns a numerical probability to key recovery or to a Poseidon2 collision or second preimage.

The position condition matters. A different direction-bit sequence denotes a different Merkle position. Equality of two roots reached at different positions is not, by itself, a collision witness for one node call. The reduction therefore requires the same position before it derives a concrete node collision. This avoids an invalid global injectivity assumption for a compressing hash.

7.2 First fraudulent spend after public observations

A fixed victim does not cover an attacker who first observes other proofs. The module `V5AdaptiveObservedTheftGame` models an arbitrary finite, ordered history of public proof records. The attacker sees the public projection of that history, chooses a spend attempt, and may depend on the entire observed prefix. The success event is the first fraudulent spend selected in this experiment.

The theorem `adaptive_first_fraudulent_spend_implies_mathematical_failure` repeats the five-way classification with a history-dependent extractor. Theorems `adaptive_first_fraudulent_spend_subset_total_final_failure` and `deployed_adaptive_first_fraudulent_spend_subset_final_or_setup` connect that classification to the final failure ledger and retain a separate event for an invalid or ambiguous victim setup.

This is a finite-history theorem, not an unlimited-attempt guarantee. If a caller permits several final attack attempts, it must bound their number or prove an appropriate state-restoration statement. It must also show that the soundness and extraction premises hold for the distribution induced by the observed history.

7.3 Raw theft accounting

For a submitted accepted proof, the attacker has already completed the work search. The raw accounting must therefore omit the 37-bit normalization used in the budget theorem of Theorem 6.2. Lean proves the conditional inequality

$$\Pr[\text{first fraudulent spend}] \le 2^{-75} + B_{\text{external}} + \Pr[\text{invalid victim setup}] \tag{11}$$

as `deployed_adaptive_first_fraudulent_spend_probability_le_two_pow_neg_75`. The external budget in this theorem is a premise; it includes production, primitive, credential, and runtime terms rather than proving them small.

A later width-19 theorem substitutes checked arithmetic for the raw batching term. Its conclusion has the form

$$\begin{aligned}\Pr[\text{first fraudulent spend}] \le 2^{-70} &+ B_{\text{nonterminal statement}} + \Pr[\text{hash/Merkle residual}] \\ &+ B_{\text{crypto}} + B_{\text{credential}} + B_{\text{runtime}} + \Pr[\text{invalid victim setup}].\end{aligned} \tag{12}$$

The exact theorem is `deployed_qm31_adaptive_theft_probability_le_two_pow_neg_70_plus_remaining`. It is an accounting result with visible remaining terms. It is not a numerical deployed theft bound.

7.4 Spent-marker transition

The source-shaped state model distinguishes account validation, proof acceptance, a recheck of mutable pool state, spent-marker creation, and the pool-root update. A successful transition requires five accounts in the specified roles, the expected program owners and writable flags, the expected derived marker address, derivation bump 255, a usable marker account, and the current pool anchor and sequence number.

The model represents two permitted pre-spend marker states. A fresh system-owned empty account may be prepared for the marker, and an already prepared empty program-owned marker may be written. A marker that already contains a spent record is rejected. On success, the transition records the nullifier, writes the new pool anchor, and increments the sequence number.

The principal results are:

- `v5_source_success_implies_requirements`: every modeled success satisfies all account, address, bump, proof, marker, and mutable-state checks;
- `v5_source_success_exact_state`: the returned state is exactly the modeled marker and pool update;
- `v5_source_rejects_preexisting_marker`: an existing marker cannot be overwritten;
- `v5_source_success_exact_trace`: a successful trace ends with the mutable-state recheck, marker write, and pool write; and
- `no_sequential_success_for_the_same_nullifier`: two modeled sequential successes cannot use the same nullifier.

The address case is slightly stronger than a replay-only statement. If two different nullifiers derive to the same marker address, the first marker still occupies the account and the second transition cannot overwrite it. Such an address alias remains a named failure in the full reduction because the connection between production address derivation and the abstract model is a separate assumption.

The marker prevents a second successful spend after the first has committed. It cannot prevent the first successful spend from being fraudulent. That is why extraction, credentials, hash binding, and the first-fraudulent-spend game remain necessary.

7.5 Close, refund, rollback, and finality

Proof-account closing is modeled as a later transaction. The close path checks the proof and refund account owners, signer and writable flags, distinct addresses, an open proof account, and a positive balance. On success it invalidates the proof account, credits the entire modeled proof balance to the supplied refund account, and leaves the pool and marker state unchanged. This is proved by `close_success_refunds_exact_recipient_and_preserves_spend`. The ordering theorem `v5_then_close_trace_has_state_writes_before_refund` places the committed marker and pool writes before the later proof invalidation and refund.

The equation `rejected_outcome_rolls_back_in_model` is true by the definition of the abstract visible-state projection. It does not prove Solana rollback. Likewise, the Lean transition does not prove account locks, system-program behavior, or persistence at finalized commitment. These are fields of `ProductionRuntimePremises`; their negations appear as named events in the final ledger.

At the source boundary, `ExactCurrentRustStateEquality` and `ExactRustCloseEquality` remain the exact equalities needed to transfer the source-shaped transition theorems to all successful

Table 9: Theft and state results.

Purpose	Lean theorem or definition
Five-way fixed-victim split	`fixed_victim_attack_implies_mathematical_failure`
Eight-term chain-level union bound	`deployed_attack_measure_le_eight_failures`
Observed-history classification	`adaptive_first_fraudulent_spend_implies_mathematical_failure`
Raw 75-bit conditional accounting	`deployed_adaptive_first_fraudulent_spend_probability_le_two_pow_neg_75`
Width-19 raw conditional accounting	`deployed_qm31_adaptive_theft_probability_le_two_pow_neg_70_plus_remaining`
Exact modeled spend state	`v5_source_success_exact_state`
Existing marker rejection	`v5_source_rejects_preexisting_marker`
Close and refund behavior	`close_success_refunds_exact_recipient_and_preserves_spend`

production runs. Recorded mainnet state supports these equalities for one execution, but one execution is not their universal proof.

8 Privacy and zero knowledge

The public transaction is intended to reveal the current anchor, nullifier, output commitment, output anchor, asset identifier, fee, deployment domain, and the proof data needed for verification. The private input note, its opening, note-tree path, output opening, and internal trace should not be learned from that view. The maintained formal results cover the algebraic hiding argument, but they do not yet establish zero knowledge for the full production transaction.

8.1 Mathematical hiding result

The mathematical model splits the masking argument into four parts: the main trace, a copied consistency component, a boundary component, and an auxiliary component used to satisfy a final linear condition. Uniform masks are drawn from finite spaces. Surjective linear maps show that the visible masked values have distributions independent of two compatible private witnesses. The auxiliary sampler is modeled at the level of whole 32-bit words, including its first-success rule and sixteen-word rejection limit for each M31 limb.

For a fixed schedule and compatible witnesses, Lean proves equality of the joint modeled view laws. The concrete theorem `encoded_concrete_fixed_schedule_view_is_witness_independent` includes the exact degree-four tower, limb order, successful whole-word sampler, kernel correction, downstream encoding, and an arbitrary deterministic output encoding. The more general theorem `encoded_modeled_view_is_witness_independent` proves the corresponding statement through an abstract finite linear-algebra interface.

An arbitrary deterministic encoding preserves equality of two distributions. This fact is useful, but the encoding argument does not identify that function with the production Rust serializer.

8.2 Retry and public selection

The prover may try a bounded number of masking schedules and release the least-indexed successful choice. Selection can leak information if the success event or retry distribution depends on the witness.

The model therefore includes the retry controller rather than treating it as an implementation detail.

The theorem `retry_selection_preserves_modeled_hiding` proves that the cap-17, least-successful-choice procedure preserves witness independence when every releasable successful branch has the same law for both witnesses. `retry_public_release_preserves_modeled_hiding` proves the same result after projecting to the selector and selected public view. To use this result for production, the premise `ExactProductionRetryReleaseCorrespondence` must identify the real proof-or-abort law, entropy consumption, selection rule, and release boundary with the modeled law. The conditional theorem `production_retry_release_hiding_of_exact_correspondence` makes that dependence explicit.

8.3 Complete byte inventory

The deployed-view model separates three arrays: a 169-byte instruction, a 40-byte sealed-proof header, and a 75,358-byte proof body. Their concatenation has 75,567 bytes. Lean proves that every offset belongs to exactly one field or semantic class, that the split and concatenation functions are inverse, and that the released selector is at the stated proof-body position.

These results are an inventory. They prevent an argument from ignoring an unclassified proof byte, but they do not show that a byte class is witness-independent. The structure `DeployedPublicView` also does not contain every fact visible through the chain. Transaction messages, upload timing, balances, logs, and unrelated account history require a wider public context if they are included in the claimed view.

8.4 Steps from the model to production

The bridge uses a sequence of separately named comparisons. On each side of the two-witness experiment it accounts for:

1. the Fiat–Shamir compiler;
2. the full Rust transcript driver;
3. the SHA-256 implementation, collision, preimage, and random-oracle steps;
4. exact serialization and parsing;
5. the five salted Merkle commitments and openings;
6. the Poseidon2 implementation, collision, and preimage steps;
7. the entropy expander and rejection sampler;
8. independence between the domain-separated random sources;
9. bounded retry and public selection; and
10. equality between the final Rust algebraic view and the mathematical view.

The structure `SideError` assigns one nonnegative error to each non-exact step. The theorem `one_side_close_to_mathematical` sums those errors by the triangle inequality. If the two mathematical laws are equal, `deployed_views_close_of_mathematical_hiding` proves

$$\Delta(V_0, V_1) \leq B_{\text{left}} + B_{\text{right}}, \tag{13}$$

Table 10: Privacy results and open production steps.

Result	Scope
`encoded_concrete_fixed_schedule_view_is_witness_independent`	Equal encoded laws for the concrete finite fixed-schedule model
`retry_selection_preserves_modeled_hiding`	Witness independence through bounded modeled retry and selection
`released_instruction_header_and_proof_byte_count`	Exact 75,567-byte modeled public-vector size
`deployed_views_close_of_mathematical_hiding`	Conditional two-sided hybrid inequality with every error supplied
`mathematical_hiding_alone_does_not_imply_deployed_hiding`	Formal counterexample to an unsupported model-to-production inference
Open work	Full Rust and serializer equality, production entropy and retry, adaptive commitment simulation, suitable computational primitive bounds, and the complete claimed public context

where Δ is the statistical distance used by the Lean model.

This formulation also exposes a metric issue. Ordinary computational assumptions about SHA-256 do not imply a small statistical distance between full byte distributions. A production claim must either state computational indistinguishability for an adversary's output or justify why each statistical-distance premise is appropriate. The current development does not make that conversion.

The transcript step can become exact once `ExactRustV5TranscriptDriverEquality` is proved; `transcript_law_matches_source_of_exact_driver` then gives equality of the output laws. The fixed-function salted-Merkle result `fixed_function_merkle_commitment_step` supplies an idealized commitment comparison from a stated salt-hiding premise. It is not yet the adaptive, oracle-relative commitment theorem required for the deployed Fiat–Shamir experiment.

8.5 Current claim

The counterexample `mathematical_hiding_alone_does_not_imply_deployed_hiding` is deliberate: two ideal point distributions can be equal while two unrelated deployed point distributions differ. It formally prevents the mathematical theorem from being presented as a production result without the connecting premises.

The current defensible statement is therefore:

> The modeled, fixed-schedule algebraic view is witness-independent under its finite sampling and linear-algebra premises. The same would hold for the specified deployed view if the named source, transcript, serialization, commitment, entropy, retry, primitive, compiler, and runtime comparisons were proved with suitable error bounds.

No theorem currently proves that the deployed transaction leaks nothing beyond its stated public view. Known-answer tests for SHA-256 or Poseidon2 show agreement on selected inputs only; they do not establish the primitive assumptions or the complete production distribution.

9 Proof design and lessons

Formalizing this verifier exposed several places where a plausible paper argument was too weak for the deployed protocol. This section records the resulting proof choices. They are useful both for checking the present work and for extending it.

9.1 Model only what the verifier receives

The verifier does not receive complete committed tables. It receives roots, selected values, salts, and compressed path material. A model that begins with full tables can prove a sound polynomial-commitment theorem while skipping the parser and authentication problem faced by the program.

The V5 model therefore starts its Merkle bridge with opened records and the literal path stream. It proves that successful parsing consumes the exact stream, reconstructs the supplied root, and returns the exact unused suffix. The outer driver must pass that suffix to the next tree and reject a nonempty final suffix. This is what makes omitted, duplicated, reordered, and trailing path material visible in the theorem statement.

9.2 Follow one candidate through all four folds

List decoding may produce several candidates. It is invalid to choose a new candidate after seeing each later fold challenge unless the probability argument pays for that adaptive choice. The proof instead chooses one member of the initial list and follows its four exact folds. If this cannot be done, the run enters one of four named bad-challenge sets. The candidate choice and the bad sets are functions of the transcript prefix available at that point.

This causal form also applies to relation messages. A prover response may depend on earlier challenges, but not on a challenge that the transcript has not squeezed. The finite counting lemmas are stated for such prefix-dependent strategies rather than for a fixed polynomial selected before the protocol.

9.3 Keep algebraic and authentication arguments separate

Low-degree agreement does not show that the queried value came from a committed table. Merkle authentication does not show that the authenticated values satisfy a low-degree relation. The Lean development joins these claims only through an explicit consumed-value map. Every FRI and relation read has to point to the value part of an accepted record in the five-tree forest.

The same separation applies to public fields. Equality of the final relation polynomial at sampled points is not enough unless the coefficients were formed from all nineteen lanes in the production order and those lanes bind the public statement. The four coefficient folds, four verifier-weight folds, and public-field rows are consequently proved as separate identities.

9.4 Do not assume a compressing hash is injective

SHA-256 and Poseidon2 map larger input spaces to smaller output spaces. A theorem cannot use global injectivity for either function. Deterministic binding statements are instead phrased as a case split: equal outputs come from equal reached inputs, or the execution supplies a concrete collision or second-preimage witness. Probability enters only when an external assumption bounds that event.

Merkle trees need an additional positional condition. Two different paths to one root expose a node collision when their leaves occupy the same position. Paths with different direction bits can describe different leaves in one valid tree. The application theorem therefore proves the same-position case and keeps alternative positions in the attack model until ownership and statement binding rule them out.

9.5 Treat the transcript as bytes and operations

An abstract list of challenges cannot detect a missing root, a swapped salt, or a work nonce absorbed at the wrong time. The transcript model records each absorb label and byte payload, each squeeze kind, and each work check. The work check is a predicate on the current state and nonce; only after it succeeds is the nonce absorbed. The source-shaped schedule proves the exact order and maps every later verifier input back to a named squeeze or decoded field.

This deterministic result is still different from a Fiat–Shamir security argument. The latter needs a model of the hash oracle, an adversarial query budget, and a theorem connecting the byte transcript to that experiment. Digest length and known-answer tests do not provide those facts.

9.6 Use one probability space and one ledger

Combining bounds from separate informal experiments can hide a conditioning change. The final soundness theorem uses one finite law and one accepted-false event. Every failure is a set in that same sample space. The nineteen-entry ledger has a proved order, no duplicates, and a proved union equality with the older detailed ledger.

This organization also makes the unit of the probability clear. The work-normalized theorem charges for grinding. The theft theorem for a completed accepted proof uses raw accounting. A factor of 2^{37} cannot be moved between these statements without changing the experiment.

9.7 Make source obligations small and exact

"The Rust verifier implements the model" is too broad to audit and too easy to assume. The implementation bridge instead names equalities for actual functions: the transcript driver, one Merkle-section helper, the five-section Merkle caller, the relation caller, the spend transition, and the close path. Focused Charon/Aeneas output is used for arithmetic, parsing, and buffer helpers that fit its supported Rust subset [22, 23].

When an enclosing Solana function cannot yet be translated, the source-shaped Lean theorem is still useful, but its final Rust equality remains in the public status table. Renaming that premise as an "accepted execution connection" would not prove it. Tests supply concrete counterexample coverage and byte traces; they do not replace the universal equality.

9.8 Separate proof, evidence, and trust

The project records a source commit, pinned build inputs, binary hash, proof hash, statement hash, transaction, and before/after state. These records answer whether one archived program and proof can be identified and replayed. They do not show that all accepted inputs satisfy the mathematical relation.

Conversely, a Lean theorem can quantify over every modeled input while depending on the correctness of Lean, mathlib, a cited decoding theorem, and cryptographic assumptions. Neither kind of evidence subsumes the other. The final report therefore uses four labels:

Checked in Lean
: A theorem with the displayed hypotheses has been accepted by the stated Lean environment.

Conditional
: The conclusion is proved once named mathematical, source, primitive, or runtime premises are supplied.

Tested or recorded
: A finite collection of executions, hashes, or chain observations agrees with the stated result.

Table 11: Archived V5 artifact identities.

Artifact	Bytes	SHA-256
Solana program	1,258,496	`4cf3c1d5ddd47efa68875c0070247e007083c5c9bb2d5988db0d644a609edf40`
Proof body	75,358	`330414df587974684643a6062d092db0519d746f0c7efe4ed2108775b685feaf`
Public statement	768	`0cdc34bc7f835640cff76d1085df9ba966df9f39eb228f3002f927cf30958113`
Signed transaction wire	—	`d5e78bb615a18274f36964248197f116a244391a77a2598f1a733c8fac7fa0b2`

Assumed No proof in the current repository supplies the claim.

The main theorem is therefore reported together with its experiment, decomposition, source locations, and trusted base. The implementation and chain connections are stated separately and are not inferred from the mathematical proof.

10 Evaluation and recorded execution

The evaluation has two purposes. First, it measures the size and audit shape of the formal and extracted source. Second, it identifies the exact program, proof, statement, and transaction used for the recorded Solana execution. Neither purpose is a substitute for the conditional security theorems.

10.1 Formal source

At commit `45d0ec466133bba6b7c52a39a288809a4943abf2`, the maintained directory `AspisFormal/AspisFormal` contains 221 Lean files and 102,832 lines. A textual count finds 3,269 declarations beginning with `theorem`. The selected Aeneas directories contain 128,429 lines of Lean, including generated translations, support libraries, tests, and proofs. The Rust files under `crates` and `programs` contain 128,708 lines.

These are repository-size figures, not proof-quality scores. Generated code can be long while proving a narrow equality, and a short theorem can depend on substantial earlier work. The theorem index in Section C therefore identifies the results used by the paper, while the dependency map in Section B shows how they are composed.

The principal theorem files contain `#print axioms` commands. In the recorded Lean environment, the selected results report only `propext`, `Classical.choice`, and `Quot.sound`. There are no project-specific axioms in those outputs [16, 28]. This check says that Lean did not accept an extra axiom through the theorem declaration; it does not validate the compiler, the cited decoding result supplied as a premise, or any cryptographic assumption.

10.2 Program and proof identities

The V5 mainnet record identifies the following artifacts.

The program identifier is `7Q2nGsPg8rbjdxKHK4jxTgEWLTyd9o1X4KMSjCieRmue`. The clean build source commit is `06788d44d30ea8cbd391899dddaf6f0acc6e4a3f`; its tree is `9b6bdfddb3c213addc2bb705c8130cce4fb2c351`. The archived build provenance records `solana-cargo-build-sbf2.3.0`, platform tools 1.48, and Rust 1.84.1. A clean build from those recorded inputs produced a binary with the same byte count and SHA-256 digest.

The build identity is strong evidence that the archived source produced the archived program. It still trusts the operating system, build scripts, Cargo dependency resolution as captured by the lock file, Rust and LLVM, Solana platform tools, the SHA-256 utility used for comparison, and the accuracy of the provenance capture.

10.3 Finalized Solana execution

The spend transaction has signature `EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENorcQtsnQj1orFmH1TPsgdbR3vJ2fE`. It finalized at slot 435,019,536. Exact signed-wire simulation and the landed transaction both report 1,334,452 compute units, below the instruction limit of 1,356,912. The derived nullifier account used bump 255.

The archived before/after records show the pool state change and creation of the spent marker. A later transaction closed the retained proof account and refunded 525,660,961 lamports at slot 435,019,649. The program data was later closed at slot 435,019,804. Thus the program and temporary live accounts are historical artifacts rather than a currently callable deployment.

The independent RPC archive stores the responses used to reconstruct the transaction and account history after those closures. Its index binds each response to a request and digest. This makes the recorded execution checkable without relying on the continued existence of the live accounts. It still relies on the correctness and completeness of the captured RPC responses and the chain history they report.

10.4 What the execution shows

The transaction establishes that one 75,358-byte proof passed the deployed program within the recorded compute limit and produced the recorded state transition. The matching simulation and landed cost give a useful check on the compute model. The later close confirms the recorded retained-proof lifecycle for this run.

It does not establish any of the following universal statements:

- every accepted byte string decodes as the Lean transcript or Merkle model;
- every accepted algebraic proof has an extracted spend witness;
- the primitive collision, preimage, or random-oracle events have a stated concrete probability;
- every rejected transaction rolls back under every relevant runtime condition; or
- every production proof has the modeled privacy distribution.

Those statements belong to the source, cryptographic, and runtime parts of the theorem. The transaction record narrows which binary and execution they must describe.

10.5 Reproduction without a fresh proof build

The repository's archived release bundle contains the proof, statement, program binary, manifests, digests, chain evidence, and offline verification scripts. Checking the artifact hashes and recorded transaction therefore does not require generating a new proof or rebuilding the Lean project. A fresh program build is a separate, optional reproducibility check and should use the recorded clean source and pinned tools.

For long-running verification work, the project also keeps dated snapshots on its local build machine. Those snapshots preserve downloaded Lean packages, Charon/Aeneas material, and Rust build output. They are operational storage, not part of the public trust argument; the public artifact identities above are content hashes.

11 Conclusion

The main result is a machine-checked security decomposition for a concrete private-spend verifier. Lean puts false acceptance in one finite experiment, proves that one protocol event and eighteen named external events cover it, and proves that the compact ledger describes exactly the same union as the earlier detailed ledger. This joins a checked private-spend relation, released circle-code and four-fold FRI arguments, transcript and five-tree Merkle models, theft reductions, and state semantics without assuming that their failure events are independent.

The 2^{-100} result is a conditional budget consequence of that decomposition. For the work-normalized experiment, checked arithmetic places the protocol budget below $0.7 \cdot 2^{-100}$; if the remaining probabilistic terms fit $0.3 \cdot 2^{-100}$, their sum fits 2^{-100}. Deterministic source and runtime correspondence are stated separately. The raw companion statement exposes its oracle-query budget and, for theft after a completed grind, retains a 2^{-70} term plus explicit remaining terms.

The main remaining project-specific step is end-to-end code verification: prove that every accepting production execution supplies the exact transcript, five authenticated openings, relation values, final polynomial, public inputs, and state transition used by Lean. Lean already supplies the prefix-timed four-way FRI reduction and adaptive fresh-oracle count. Published decoding and BCS Fiat–Shamir results, together with standard primitive, compiler, proof-assistant, and runtime assumptions, can remain stated assumptions.

The archived program, proof, statement, build provenance, and finalized transaction make the implementation being discussed identifiable. The Lean source makes the mathematical core mechanically checkable, and the focused Rust translations already cover many of the small functions used by the verifier. The result is not whole-program verification, but it gives a precise, checkable account of what is proved and the exact equalities needed to complete the production connection.

A Assumptions and remaining verification

Most of the mathematical core is checked in Lean. The main project-specific gap is end-to-end production-code correspondence. Published decoding and Fiat–Shamir results, primitive security, and toolchain/runtime correctness are the standard stated inputs for the concrete instantiation.

A.1 End-to-end code connection

The main open theorem is that every accepting production verifier run produces exactly the values consumed by the Lean spend model. This includes parsing, all nineteen relation columns, public inputs, the transcript, authenticated openings, folds, final polynomial, and the state callback. Lean already proves the spend relation once those values have been extracted.

The production proof is divided into focused equalities for the enclosing transcript driver, one Merkle-section helper, the five-section Merkle caller, the FRI consumer, the relation caller, the spend transition, and the close path. Small arithmetic, parsing, buffer, transcript, and Merkle functions already have source-shaped or Charon/Aeneas proofs. The exact remaining function list is in Section E.

A.2 How the assumptions enter the theorem

The audit ledger retains source and runtime mismatches as named events so that none can disappear from the proof. The conventional cryptographic statement instead assumes the twelve deterministic production correspondences and assigns probability only to the protocol, primitive, and credential events. Under those correspondences, the deterministic event sets are empty and take budget zero. This is the specialization in Equation (4); it does not assign a fictional tiny failure probability to parser correctness or rollback semantics.

A.3 V5 soundness status

Lean checks the exact circle domains, encoder order, degrees, distances, local radix-four folds, prefix-conditioned bad sets, compatible candidate chain, relation reductions, query calculation, adaptive fresh-oracle count, and final probability arithmetic. This proves the V5-specific four-way FRI part rather than assuming that the different S-two folding protocol applies unchanged.

The final event interface identifies the event extracted from production with the event covered by the published BCS state-restoration theorem. Its remaining inputs are the production transcript equality and the stated BCS and SHA-256 random-oracle assumptions. They are part of the code connection and standard cryptographic model, not a separate unfinished V5 FRI theorem.

The 2^{-100} theorem is a work-normalized conditional budget theorem. The resource-bounded form in Equation (8) and the raw theft theorem keep the completed-work factor out and report their remaining terms separately.

A.4 Standard trusted inputs

The development uses the following normal assumptions:

- correctness of the published circle-code decoding result;
- correctness of the published BCS state-restoration and Fiat–Shamir soundness result for the modeled public-coin IOP;
- collision, target-preimage, and modeled random-oracle security for SHA-256 where each property is used;
- the stated Poseidon2 implementation and security properties;
- correctness of Lean, mathlib, Charon/Aeneas for the translations used, Rust, LLVM, and the Solana build tools; and
- the stated Solana behavior for address derivation, account ownership, writable locks, rollback, committed persistence, and finalized state.

These assumptions need to be named and given suitable parameters in a concrete security claim, but they do not normally need to be reproved inside the Aspis development. Known-answer tests and reproducible builds support implementation identity; they are not used as proofs of collision resistance.

A.5 Privacy scope

The mathematical hiding theorem proves witness independence for its complete finite modeled view. Extending it to a deployed zero-knowledge claim requires the production masking, entropy, retry, commitment, serialization, and public observation model. This is additional work only if the paper claims full deployed zero knowledge; it is not needed for the private-spend soundness theorem.

A.6 Status summary

Table 12: Current proof status.

Area	Status	Remaining work or assumption
Spend relation after extraction	Checked in Lean	Faithful deployed Poseidon2 functions are supplied
Circle domains and encoders	Checked in Lean	Published decoding result is used as a stated theorem
Four-round FRI and candidate chain	Checked in Lean	Published BCS Fiat–Shamir theorem and SHA-256 random-oracle assumption apply once the production equality is supplied
Transcript, Merkle, and relation source models	Checked in Lean	Enclosing production Rust equalities remain
Work-normalized 2^{-100} budget composition	Checked in Lean	Exact event connection and six probabilistic external bounds are supplied; deterministic correspondences take budget zero
Spent-marker and refund model	Checked in Lean	Production Rust equality and standard Solana behavior are supplied
Source, binary, and finalized execution	Reproduced and recorded	Relies on the recorded build and chain tools
Production zero knowledge	Separate extension	Requires the production prover and complete public-view connection

B Dependency map

The formal development is easier to review by following the direction of the main implication rather than the repository's alphabetical file list.

Table 13: Main formal dependencies.

Layer	Principal modules	Result passed to the next layer
Field and group	`V5M31RawMulReduction`, `V5M31RawSubNeg`, `V5M31InverseAdditionChain`, `V5ComponentCQM31TowerExact`, `CircleGroupOrder`	Exact base and extension arithmetic, circle order, and distinct domain points
Spend relation	`ArithmetizationCore`, `ValueConservation`, `V5AcceptedSpendRelation`	Normalized arithmetic, hash, and Merkle rows imply a complete spend witness

Layer	Principal modules	Result passed to the next layer
Encoders	`V5FriInitialCircleEncoderIdentity`, `V5FriConcreteEncoderApplicability`, `V5Width19S2ApplicabilityAudit`	Exact stored evaluation order, polynomial degrees, distances, and checked parameters for the cited decoding proposition
Four-round extraction	`V5FriReleasedAdaptiveExtraction`, `V5FriRelationCandidateBridge`, `V5AdaptiveSumcheckChallengeBound`	One coherent initial candidate or a counted fold/relation failure
Transcript	`V5TranscriptConnection`, `V5TranscriptSourceAdapter`, `V5CryptographicAssumptions`	Exact modeled byte schedule, consumed challenges, and named primitive/source failure events
Authenticated openings	`V5MerkleAuthenticationBinding`, `V5MerkleRustBridge`, `V5MerkleConsumedValueBridge`	Five accepted paths, exact parsed values, and authenticated FRI reads outside the SHA-256 collision event
Production relation	`V5RelationStressSourceBridge`, `V5AcceptedExecutionSecurityBridge`	Source-shaped relation results and explicit outer Rust equalities needed for `AcceptedRunExtractsTrace`
Soundness experiment	`V5FinalSecurityAccounting`, `V5HundredBitSecurityMargin`, `V5UnifiedSecurityExperiment`	One finite accepted-false experiment, exact failure ledger, and conditional work-normalized 2^{-100} theorem
Theft and state	`V5FixedVictimTheftGame`, `V5AdaptiveObservedTheftGame`, `V5TheftStateTransitionReduction`, `V5ProductionStateBridge`	First-fraudulent-spend classification and exact modeled marker, pool, close, and refund behavior
Privacy	`V5DeployedZeroKnowledgeBridge` and its finite hiding dependencies	Mathematical witness independence plus an explicit list of production and cryptographic comparisons

The arrows in this map are not all unconditional. The encoder layer uses the cited decoding result, the transcript and opening layers retain enclosing Rust equalities, the probability layer retains event bounds, and the state layer retains Solana runtime premises. Their names are part of the formal interfaces.

C Theorem index

The following table gives the principal Lean declarations used in the report. Each name is paired with its source module so that it can be found without a line-number reference.

Table 14: Principal checked declarations.

Module	Declaration and role
`ValueConservation`	`value_conservation_no_wraparound`: converts the field balance equation to integer value conservation under the range assumptions.
`V5AcceptedSpendRelation`	`extracted_trace_implies_spend_relation`: extracted residual, hash, and path rows imply the complete spend relation.
`V5AcceptedSpendRelation`	`false_accept_event_subset_bad_event` and `false_accept_measure_le_bad_event`: conditional accepted-run extraction gives pointwise and measure forms of false-acceptance containment.
`CircleGroupOrder`	`orderOf_g`: the released M31 circle generator has order 2^{31}.
`V5FriInitialCircleEncoderIdentity`	`encoder0_eq_stored_circle_eval`: the initial encoder equals evaluation of the exact circle polynomial in released storage order.
`V5FriConcreteEncoderApplicability`	`concrete_line_encoder_distances`: the four later code distances follow from exact polynomial agreement bounds.
`V5Width19S2ApplicabilityAudit`	`published_appendixA2_supplies_exact_width19_curve_decodable`: released parameters instantiate the stated external decoding proposition.
`V5FriReleasedAdaptiveExtraction`	`accepted_ideal_fri_extracts_initial_candidate_or_counted`: an accepted ideal run yields one initial candidate through all folds or a counted event.
`V5FriForwardCompatibleChain`	`accepted_ideal_fri_restores_complete_chain_or_prefix_bad`: the released four-fold chain is restored or a bad set fixed by the proper prefix occurs.
`V5ForwardAcceptedFalseRawAccounting`	`ReleasedIdealAcceptedFalseExperimentData.toEvents_coverage`: the accepted-false ideal experiment is covered by the released FRI and relation events.
`V5ForwardAcceptedFalseRawAccounting`	`adaptiveCompatibilityReleasedFriFailureProbability_le_queryBudget_mul`: adaptive fresh attempts cost at most the query budget times the one-attempt FRI bound.
`V5FiatShamirAdaptiveQueryBudget`	`ProductionAdaptiveFreshOracleConnection`: exact remaining production SHA-256 transcript, query-budget, and ideal fresh-oracle comparison.
`V5FriRelationCandidateBridge`	`evaluation_discrepancy_eq_folded_difference`: the final evaluation discrepancy equals the folded candidate difference.

Module	Declaration and role
`V5TranscriptConnection`	`source_shaped_helper_composition_exact`: the source-shaped helpers produce the complete ordered transcript trace.
`V5TranscriptConnection`	`complete_schedule_checks_each_work_immediately_before_absorb`: every work nonce is checked directly before its absorb operation.
`V5TranscriptConnection`	`source_consumption_uses_exact_squeezes`: later verifier inputs are the values obtained from the named squeeze positions.
`V5TranscriptSourceAdapter`	`source_adapter_driver_is_exact`: decoded source fields produce the same pure transcript-driver result.
`V5TranscriptSourceAdapter`	`source_adapter_passes_exact_values_to_fri_and_relation`: the adapter passes the modeled challenge values to their consumers.
`V5BoundedQuerySamplerUniformity`	`deployed_q18_conditioned_uniform`: the successful bounded sampler is uniform on ordered eighteen-element schedules without replacement.
`V5WithoutReplacementQuerySoundness`	`ideal_miss_probability_eq_descFactorial_ratio`: exact miss probability for a fixed bad set.
`V5MerkleRustBridge`	`fixed_record_eq_value_append_salt`: every exact opened record is value bytes followed by the 32-byte salt.
`V5MerkleRustBridge`	`exactSection_frontier_has_no_omission_or_trailing`: the exact section accounts for all frontier material.
`V5MerkleRustBridge`	`exactV5Run_yieldsForest`: five successful modeled sections construct an accepted forest.
`V5MerkleRustBridge`	`productionAcceptedForest_values_unique`: production values are unique under fixed roots and queries outside a SHA-256 collision, assuming the focused source equality.
`V5MerkleConsumedValueBridge`	`rustObservation_exposes_only_authenticated_fri_values`: the specified opening/FRI observation uses only values authenticated by the forest.
`V5UnifiedSecurityExperiment`	`final_failure_partition_is_exact`: the six protocol and eighteen external entries partition the old twenty-four-entry ledger.
`V5UnifiedSecurityExperiment`	`ordered_unified_failure_kinds_are_exactly_once`: the nineteen new kinds are present once each.
`V5UnifiedSecurityExperiment`	`total_unified_failure_eq_total_final_failure`: old and new failure unions are equal.

Module	Declaration and role
`V5UnifiedSecurityExperiment`	`accepted_false_probability_le_core_plus_external_sum`: one core event plus eighteen external event probabilities bounds false acceptance.
`V5HundredBitSecurityMargin`	`corrected_selected_release_core_le_seven_tenths`: conditional released core arithmetic fits $0.7 \cdot 2^{-100}$.
`V5UnifiedSecurityExperiment`	`work_normalized_accepted_false_probability_le_two_pow_neg_100_of_released_core`: conditional work-normalized budget implication.
`V5FixedVictimTheftGame`	`fixed_victim_attack_implies_mathematical_failure`: five-way fixed-victim attack classification.
`V5FixedVictimTheftGame`	`deployed_attack_measure_le_eight_failures`: adds address, runtime/state, and victim-setup events.
`V5AdaptiveObservedTheftGame`	`adaptive_first_fraudulent_spend_implies_mathematical_failure`: extends the mathematical split to an observed finite proof history.
`V5AdaptiveObservedTheftRawAccounting`	`deployed_adaptive_first_fraudulent_spend_probability_le_two_pow_neg_75`: conditional raw accounting with external and setup terms.
`V5AdaptiveTheftWidth19Bound`	`deployed_qm31_adaptive_theft_probability_le_two_pow_neg_70_plus_remaining`: raw width-19 term plus all remaining terms.
`V5TheftStateTransitionReduction`	`v5_source_success_exact_state`: exact marker and pool state after a modeled success.
`V5TheftStateTransitionReduction`	`v5_source_rejects_preexisting_marker`: an occupied marker cannot be overwritten.
`V5TheftStateTransitionReduction`	`close_success_refunds_exact_recipient_and_preserves_spend`: exact modeled close and refund behavior.
`V5DeployedZeroKnowledgeBridge`	`encoded_concrete_fixed_schedule_view_is_witness_independent`: concrete finite fixed-schedule hiding after deterministic encoding.
`V5DeployedZeroKnowledgeBridge`	`deployed_views_close_of_mathematical_hiding`: two-sided conditional hybrid bound.
`V5DeployedZeroKnowledgeBridge`	`mathematical_hiding_alone_does_not_imply_deployed_hiding`: counterexample showing why production comparisons are required.

D How to read the final bound

The final result can be checked in four steps.

1. Fix one finite probability law for the complete work-normalized experiment. All prover coins, ideal-oracle values, and environment choices used by the application must be represented in that law.
2. Prove pointwise that every accepted false outcome is either in the single protocol event or one of the eighteen external events. The maintained released-execution constructor supplies this step only after its accepted-execution classification is supplied.
3. Prove an event-level bound $\Pr[C] \leq B_{\text{core}}$ for the exact protocol event and discharge the released applicability premises. Checked arithmetic then gives $B_{\text{core}} \leq 0.7 \cdot 2^{-100}$.
4. Prove the deterministic production correspondences, making their twelve mismatch events empty, and bound the six remaining primitive and credential events by a total of at most $0.3 \cdot 2^{-100}$.

The union bound then gives

$$\begin{aligned}
\Pr[A_{\text{false}}] &\leq \Pr[C] + \sum_{j=1}^{6} \Pr[P_j] \qquad \text{under } D_{\text{prod}} \\
&\leq B_{\text{core}} + B_{\text{external}} \\
&\leq 0.7 \cdot 2^{-100} + 0.3 \cdot 2^{-100} \\
&= 2^{-100}.
\end{aligned}$$

No independence between the nineteen events is needed. The conclusion is no stronger than its law. If the law conditions on a completed work search, the 37-bit work factor cannot still be charged. If an application permits many attempts, it must model them or apply a justified multi-attempt bound.

The full eighteen-entry external ledger remains the audit interface. Twelve entries require deterministic correspondence proofs; the other six require concrete cryptographic or credential bounds. A single informal statement such as “standard cryptographic assumptions” does not discharge either group. Each field in `AssumedExternalSecurityBounds` refers to an event in the exact experiment, and deterministic events receive zero only after their correspondence premise has been proved.

E Production-source checklist

This checklist states what a complete source proof must establish. “Open” means that the pure or source-shaped Lean result exists but the universal equality for the named production function is not yet proved.

Table 15: Production source audit.

Production function or area	Status	Required equality
`Transcript::absorb` and `Transcript::squeeze_block`	Focused model	Exact labels, byte payloads, output hash, and state-advance hash are modeled; full all-input source equality must be inherited by the outer driver.

Production function or area	Status	Required equality
`Transcript::grinding_ok` and the six check-and-absorb helpers	Source-shaped proof and tests	Each nonce is checked against the pre-absorb state and correct difficulty, then absorbed once with its exact record.
`v5_complete_transcript_before_final_with_statement_digest` and query derivation	Open	Prove `ExactRustV5TranscriptDriverEquality`, including all returned challenges, selector, and eighteen query positions.
`verify_state_only_private_opening_from_proof_with_topology`	Open	Prove `VerifyStateOnlyPrivateOpeningWithTopologySourceEquality`: exact record slices, salt split, framed hash calls, shifted indices, slots, binary cap, frontier use, and returned suffix.
`verify_v5_private_openings_from_proof`	Open	Prove `VerifyV5DriverCompositionSourceEquality`: five calls in fixed order, each supplied with the prior suffix, with no final bytes left.
`check_v5_fri_queries` and its opening lookups	Open outer equality	Show every algebraic read is exactly the decoded value portion returned by the authenticated opening driver.
`prepare_v5_pcs_claims`, compact folds, and relation rounds	Partial	Focused arithmetic and source-shaped equalities are checked; prove the enclosing relation caller and raw-success equality.
All nineteen relation columns and public inputs	Open composition	Prove `AcceptedRunExtractsTrace`, with no omitted column, residual, fold, public field, final coefficient, hash row, or spend Merkle path.
`verify_and_apply_atomic_payment_state_traced_inner`	Open	Prove `ExactCurrentRustStateEquality`, including account roles, owners, writability, expected derived marker and bump, mutable-state recheck, marker write, and pool update.
`refund_program_owned_proof_account`	Open	Prove `ExactRustCloseEquality`, including refund-account selection, ownership, signers, writable flags, invalidation, and complete balance transfer.
Solana runtime	Assumed	Prove or assume system-program behavior, address derivation, writable locks, rollback, committed persistence, and finalized observation.

For each row, fixed-input tests are useful for finding mismatched offsets, labels, endianness, and failure behavior. Completion requires a statement quantified over every accepted input in the stated function domain.

F Archived-execution checklist

An independent check of the historical execution should verify the following items from the archived V5 mainnet bundle.

1. Check the bundle's signed or recorded checksum list before reading the individual files.
2. Check the program, proof, statement, and signed-wire SHA-256 values in Table 11.
3. Check that the clean build record names source commit `06788d44d30ea8cbd391899dddaf 6f0acc6e4a3f`, source tree `9b6bdfddb3c213addc2bb705c8130cce4fb2c351`, and the recorded Solana/Rust platform tools.
4. Run the bundled offline verifier against the archived proof and public statement. This checks the archive format and expected fixed input; it is not a universal proof.
5. Compare program identifier `7Q2nGsPg8rbjdxKHK4jxTgEWLTyd9o1X4KMSjCieRmue` and spend transaction signature `EJviPgF12i9iK2CveVaQSMeFQqDMFPQ1iPRUYEwNQE3zGquTUZNJXPZEENor cQtsnQj1orFmH1TPsgdbR3vJ2fE` with the captured RPC responses.
6. Confirm finalized slot 435,019,536, 1,334,452 consumed compute units, the 1,356,912 unit limit, and marker-address bump 255.
7. Compare the before/after pool anchor, pool sequence, marker owner, and marker data with the public statement and the modeled transition.
8. Check the later proof-account close at slot 435,019,649 and its 525,660,961-lamport refund.
9. Record that the program data was closed at slot 435,019,804 and that the demonstration accounts were later cleaned up. A failed live account lookup today does not contradict the archived state.
10. Distinguish the deployment source commit from the paper's later formal snapshot. Recheck theorem statements against the latter and artifact identity against the former.

The archive allows these checks without rebuilding the proof system. A fresh Lean build or program build answers a different reproducibility question and should use the stored dependencies and toolchain rather than downloading or regenerating them without need.

References

[1] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017. doi: 10.1145/3133956.3134078. URL `https://doi.org/10.1145/3133956.3134078`.

[2] José Bacelar Almeida, Manuel Barbosa, Karim Eldefrawy, Stéphane Graham-Lengrand, Hugo Pacheco, and Vitor Pereira. Machine-checked ZKP for NP-relations: Formally verified security proofs and implementations of MPC-in-the-Head. In *Proceedings of the 2021 ACM SIGSAC*

Conference on Computer and Communications Security, pages 2587–2600. ACM, 2021. doi: 10.1145/3460120.3484771. URL `https://doi.org/10.1145/3460120.3484771`.

[3] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon proximity testing with super-fast verification. In *Advances in Cryptology–EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025. doi: 10.1007/978-3-031-91134-7_8. URL `https://doi.org/10.1007/978-3-031-91134-7_8`.

[4] Dominic Barker. Aspis source and formal development. Source repository, 2026. URL `https://github.com/DJBarker87/aspis/tree/45d0ec466133bba6b7c52a39a288809a4943abf2`. Paper snapshot `45d0ec466133bba6b7c52a39a288809a4943abf2`.

[5] Gilles Barthe, Juan Manuel Crespo, Benjamin Grégoire, César Kunz, and Santiago Zanella-Béguelin. Computer-aided cryptographic proofs. In *Interactive Theorem Proving*, volume 7406 of *Lecture Notes in Computer Science*, pages 11–27. Springer, 2012. doi: 10.1007/978-3-642-32347-8_2. URL `https://doi.org/10.1007/978-3-642-32347-8_2`.

[6] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE, 2014. doi: 10.1109/SP.2014.36. URL `https://doi.org/10.1109/SP.2014.36`.

[7] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography—TCC 2016-B, Part II*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016. doi: 10.1007/978-3-662-53644-5_2. URL `https://doi.org/10.1007/978-3-662-53644-5_2`.

[8] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.14. URL `https://doi.org/10.4230/LIPIcs.ICALP.2018.14`.

[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL `https://eprint.iacr.org/2018/046`.

[10] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *11th Innovations in Theoretical Computer Science Conference*, volume 151 of *Leibniz International Proceedings in Informatics*, pages 5:1–5:32. Schloss Dagstuhl–Leibniz-Zentrum f"ur Informatik, 2020. doi: 10.4230/LIPIcs.ITCS.2020.5. URL `https://doi.org/10.4230/LIPIcs.ITCS.2020.5`.

[11] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. *Journal of the ACM*, 70(5):1–57, 2023. doi: 10.1145/3614423. URL `https://doi.org/10.1145/3614423`.

[12] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michal Zając. Fiat–Shamir security of FRI and related SNARKs. In *Advances in Cryptology–ASIACRYPT 2023, Part II*, volume 14439 of *Lecture Notes in Computer Science*, pages 3–40. Springer, 2023. doi: 10.1007/978-981-99-8724-5_1. URL `https://doi.org/10.1007/978-981-99-8724-5_1`.

[13] Sarah Bordage, Alessandro Chiesa, Ziyi Guan, and Ignacio Manzur. All polynomial generators preserve distance with mutual correlated agreement. Cryptology ePrint Archive, Report 2025/2051, 2025. URL `https://eprint.iacr.org/2025/2051`.

[14] Dan Carmon, Lior Goldberg, Ulrich Haböck, Leonardo Lerer, Ilya Lesokhin, Shahar Papini, and Shahar Samocha. S-two whitepaper. Cryptology ePrint Archive, Report 2026/532, 2026. URL `https://eprint.iacr.org/2026/532`. Revision dated 24 March 2026.

[15] Elizabeth Crites and Alistair Stewart. On Reed–Solomon proximity gaps conjectures. Cryptology ePrint Archive, Report 2025/2046, 2025. URL `https://eprint.iacr.org/2025/2046`.

[16] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction–CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37. URL `https://doi.org/10.1007/978-3-030-79876-5_37`.

[17] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. Simple high-level code for cryptographic arithmetic—with proofs, without compromises. In *2019 IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019. doi: 10.1109/SP.2019.00005. URL `https://doi.org/10.1109/SP.2019.00005`.

[18] Denis Firsov and Benjamin Livshits. The ouroboros of ZK: Why verifying the verifier unlocks longer-term ZK innovation. Cryptology ePrint Archive, Report 2024/768, 2024. URL `https://eprint.iacr.org/2024/768`.

[19] Ulrich Hab"ock. A note on mutual correlated agreement for Reed–Solomon codes. Cryptology ePrint Archive, Report 2025/2110, 2025. URL `https://eprint.iacr.org/2025/2110`.

[20] Ulrich Haböck, Daniel Lubarov, and Jacqueline Nabaglo. Reed–Solomon codes over the circle group. Cryptology ePrint Archive, Report 2023/824, 2023. URL `https://eprint.iacr.org/2023/824`.

[21] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Report 2024/278, 2024. URL `https://eprint.iacr.org/2024/278`.

[22] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, 6(ICFP):711–741, 2022. doi: 10.1145/3547647. URL `https://doi.org/10.1145/3547647`.

[23] Son Ho, Guillaume Boisseau, Lucas Franceschino, Yoann Prak, Aymeric Fromherz, and Jonathan Protzenko. Charon: An analysis framework for rust. In *Computer Aided Verification*, 2025. URL `https://arxiv.org/abs/2410.18042`. Tool paper; preprint arXiv:2410.18042.

[24] Light Protocol contributors. Groth16 merkle-tree setup artifacts. Source repository, commit `85243034a19b98952440f5823f1a5ab139e52ca3`, 2026. URL `https://github.com/Lightprotocol/gnark-mt-setup/tree/85243034a19b98952440f5823f1a5ab139e52ca3`. Accessed 15 August 2026.

[25] Privacy Cash contributors. Privacy cash mainnet program account. Solana Explorer, 2026. URL `https://explorer.solana.com/address/9fhQBbumKEFuXtMBDw8AaQyAjCorLGJQiS3skWZdQyQD`. Accessed 15 August 2026.

[26] Privacy Cash contributors. Privacy cash circuit artifacts and trusted-setup record. Source repository, commit `aa6818b76b23edcf05b8b9829a9cd900fdb1d241`, 2026. URL `https://github.com/Privacy-Cash/privacy-cash/tree/aa6818b76b23edcf05b8b9829a9cd900fdb1d241`. Accessed 15 August 2026.

[27] Protocol 01 contributors. Protocol 01: Transparent shielded payments on solana. Source repository, commit `790dbff34ac96302fc6336d411fac14f683f5be9`, 2026. URL `https://github.com/IsSlashy/Protocol-01/tree/790dbff34ac96302fc6336d411fac14f683f5be9`. Accessed 15 August 2026.

[28] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824. URL `https://doi.org/10.1145/3372885.3373824`.

[29] Verified-zkEVM contributors. ArkLib: Formally verified arguments of knowledge in lean. Source repository, commit `5a9778db3cafa29288c194077a3780bf524f1137`, 2026. URL `https://github.com/Verified-zkEVM/ArkLib/tree/5a9778db3cafa29288c194077a3780bf524f1137`. Accessed 15 August 2026.

[30] Jotaro Yano. solana-pqzk-fullchain: Full on-chain verification of ZK-STARK proofs and post-quantum signatures on solana. *Journal of Open Source Software*, 11(123):9923, 2026. doi: 10.21105/joss.09923. URL `https://doi.org/10.21105/joss.09923`.

[31] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. HACL*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017. doi: 10.1145/3133956.3134043. URL `https://doi.org/10.1145/3133956.3134043`.